

Contents

01 - はじめに	3
インストール	3
プロジェクトの初期化	3
最初のプログラム	4
コメントの書き方	4
02 - 変数と型	4
変数宣言	4
@const による不変変数（定数）宣言	4
型アノテーション	5
基本型	5
文字列操作	6
エスケープシーケンス	7
f-string（文字列補間）	7
型キャスト（as）	7
再代入のルール	8
タプル分割代入	8
03 - 演算子	9
算術演算子	9
比較演算子	9
論理演算子	10
ビット演算子	10
複合代入演算子	11
インクリメント・デクリメント演算子	11
型昇格ルール	11
所属演算子	12
制御構文	12
if / else	12
case	13
while ループ	14
for ループと range	14
break と continue	15
ネストの例	16
スコープのルール	16
パターンマッチング	17
関数	18
基本的な関数定義	18
関数の呼び出し	18
再帰関数	18
関数オーバーロード	18
戻り値型の省略（Unit 型）	19
デフォルト引数	19
ラムダ関数	19
クロージャ	20
高階関数	21
UFCS（統一関数呼び出し構文）	21
演習	22
Record と列挙型	22
Record の定義	22

インスタンスの生成	22
フィールドアクセス（ドット記法）	22
フィールドへの代入	23
関数の引数としての Record	23
ネストした Record	23
列挙型（enum）	23
関連データ付き列挙型（ADT）	24
ジェネリック列挙型	25
演算子オーバーロード	25
演習	26
コレクションとイテレータ	27
タプル	27
リスト	27
マップ	30
セット	31
イテレータ	32
演習	33
エラーハンドリング	33
Option 型	33
Result 型	34
契約による設計	35
使い分けの指針	37
よくあるミス	37
演習	37
パッケージ	38
from/import 構文	38
サブディレクトリ（ドット区切り）	38
ディレクトリパッケージ	38
相対インポート	38
標準ライブラリ（std）	39
検索パスの優先順位	39
RPATH 環境変数	39
制限事項	39
並行処理	40
async/await	40
@parallel	41
OS スレッド	41
ネットワーク（TCP ソケット）	43
適切なモデルの選択	44
よくあるミス	45
演習	45
テスト	45
テストの実行	45
テストの書き方	45
マッチャー	46
出力フォーマット	47
モック	48
パラメタライズドテスト	48
プロパティベーステスト	48
契約を使ったテスト	49

制限事項	49
演習	49
プロジェクトを作る	50
プロジェクトのセットアップ	50
ステップ 1: データモデルの定義	50
ステップ 2: タスク操作	51
ステップ 3: 表示	51
ステップ 4: メインプログラム	52
ステップ 5: テストの作成	52
次のステップ	53
English 日本語 繁體中文	

01 - はじめに

次のチュートリアル -> [02 - 変数と型](#)

インストール

クイックインストール (macOS Apple Silicon)

```
curl -fsSL https://raw.githubusercontent.com/t0k0sh1/ry/main/install.sh | sh
```

ry バイナリが ~/.local/bin に、標準ライブラリが ~/.ry/share/std/ にインストールされます。

~/.local/bin が PATH に含まれていることを確認してください:

```
export PATH="$HOME/.local/bin:$PATH"
```

ソースからビルドする場合や他のプラットフォームについては、[README のインストールセクション](#)を参照してください。

プロジェクトの初期化

ry new コマンドで新しいプロジェクトを作成できます:

```
ry new my-project
cd my-project
```

これにより以下のファイルとディレクトリが生成されます:

- package.toml - プロジェクト設定ファイル
- src/main.ry - エントリポイント (サンプルコード付き)

カレントディレクトリをプロジェクトとして初期化する場合は ry init を使います:

```
mkdir my-project
cd my-project
ry init
```

詳細は[プロジェクト管理](#)を参照してください。

最初のプログラム

以下の内容を `hello.ry` というファイルに保存してください:

```
print("Hello, World!")
```

次のコマンドで実行します:

```
ry hello.ry
```

出力:

```
Hello, World!
```

パイプや Here-document を使って、`-c` フラグで標準入力からコードを実行することもできます:

```
echo 'print("Hello, World!")' | ry -c
```

```
ry -c <<'RY'
print("Hello, World!")
RY
```

コメントの書き方

から行末までがコメントとして扱われます。

```
# これはコメントです
```

```
print("Hello") # 行末コメントも使えます
```

コメントはコードの動作に影響を与えません。

次のチュートリアル -> [02 - 変数と型](#)

[English](#) | [日本語](#) | [繁體中文](#)

02 - 変数と型

<- [01 - はじめに](#) / 次-> [03 - 演算子](#)

変数宣言

Ry では、シンプルな代入構文で変数を宣言します。デフォルトでは変数は可変です。

```
x = 42          # int 型として推論
y = 3.14        # float 型として推論
flag = true     # bool 型として推論
name = "Ry"     # str 型として推論
```

@const による不変変数（定数）宣言

@const ディレクティブで不変な変数（定数）を宣言します。宣言後に値を変更することはできません。

```
@const
x = 42          # int 型として推論
```

```
@const
y = 3.14      # float 型として推論

@const
flag = true   # bool 型として推論

@const
name = "Ry"   # str 型として推論
```

型アノテーション

変数の型を明示的に指定できます。

```
x: int = 42

rate: float = 0.5

ok: bool = false

msg: str = "hello"
```

型アノテーションと実際の値の型が一致しない場合はコンパイルエラーになります。

基本型

型	説明	リテラル例
int	64 ビット整数	0, 42, -10
u8	符号なし 8 ビット整数 (0-255)	b: u8 = 42
float	64 ビット浮動小数点数	0.0, 3.14, -1.5
bool	真偽値	true, false
str	文字列	"hello", ""

低レベル数値型

Ry はメモリレイアウトを正確に制御するための低レベル数値型も提供しています。これらの型には**暗黙の変換がありません**。明示的なキャストには `as` を使用する必要があります。

型	説明	例
i8	8 ビット符号付き整数	x: i8 = 42
i16	16 ビット符号付き整数	x: i16 = 100
i32	32 ビット符号付き整数	x: i32 = 42
i64	64 ビット符号付き整数	x: i64 = 100
u8	8 ビット符号なし整数	x: u8 = 200
u16	16 ビット符号なし整数	x: u16 = 60000
u32	32 ビット符号なし整数	x: u32 = 3000000000
u64	64 ビット符号なし整数	x: u64 = 100
f32	32 ビット浮動小数点数	x: f32 = 3.14

```
a: i32 = 10
b: i32 = 20
```

```

c = a + b          # OK: i32 + i32 → i32

d = 42
# e = a + d        # エラー: i32 と int の混在はできない

y = a as int       # int への明示的キャスト
z = d as i32       # i32 への明示的キャスト

# 符号なし型は符号なし演算を使用
x: u32 = 3000000000
y: u32 = 7
q = x / y          # 符号なし除算 (UDiv)

```

注意: 低レベル整数型の / は (Rust のように) 整数除算を行い、浮動小数点除算ではありません。符号付き型は SDiv、符号なし型は UDiv を使用します。

注意: 低レベル整数型の算術演算はオーバーフロー時にラップアラウンドします。符号付き型は 2 の補数、符号なし型はモジュラー算術を使用します。オーバーフローが懸念される場合は高レベルの int 型 (64 ビット) を使用してください。

固定長配列

低レベル型向けに、Ry は固定長の連続配列 T[N] を提供しています。これらはスタック上に割り当てられ、コンパイル時にサイズが決定されます。

```

buf: i32[4] = [1, 2, 3, 4]
print(buf[0])      # 1
buf[2] = 99
print(buf[2])      # 99
print(length(buf)) # 4

pixels: u8[3] = [255, 128, 0]

```

文字列操作

文字列に対してさまざまな操作が使えます。

```

a = "Hello"
b = "World"

# 結合
c = a + ", " + b  # "Hello, World"

# 比較 (辞書順)
print(a == b)     # false
print(a != b)     # true
print(a < b)      # true ("H" < "W")

# 長さ
print(length(a))  # 5

# 部分文字列チェック
s = "Hello, World!"
print(contains(s, "World")) # true

```

```
print(starts_with(s, "Hello")) # true
print(ends_with(s, "!"))      # true
```

エスケープシーケンス

文字列中で以下のエスケープシーケンスが使えます。

シーケンス	意味
<code>\n</code>	改行
<code>\r</code>	復帰
<code>\t</code>	タブ
<code>\\</code>	バックスラッシュ
<code>\"</code>	ダブルクォート
<code>\0</code>	ヌル文字

```
print("Hello\nWorld") # 2行に分けて出力
print("A\tB")         # タブ区切り
print("say \"hi\"")    # ダブルクォートを含む文字列
```

f-string (文字列補間)

変数や式を文字列に埋め込む場合は、+ で結合する代わりに `f"..."` を使います。{} 内の式は評価されて自動的に文字列に変換されます。

```
name = "Alice"
print(f"Hello {name}") # Hello Alice

x = 3
y = 4
print(f"{x} + {y} = {x + y}") # 3 + 4 = 7
```

リテラルの波括弧を含めるには {{ と }} を使います:

```
print(f"{{escaped}}") # {escaped}
```

なぜ f-string なのか? "Hello " + name のような手動の結合よりも読みやすく、エラーが起きにくくなります。テキストと動的な値を混在させる場合は f-string を使ってください。

よくあるミス: f プレフィックスなしで "Hello {name}" と書くと、変数の値ではなくリテラルテキスト {name} が得られます。

型キャスト (as)

as で型を明示的に変換します。縮小変換 (float から int など) や数値型と非数値型の間の変換 (str, bool) には as が必要です。一部の安全な数値昇格 (式や特定の呼び出しにおける int から float など) は暗黙的に行われます。

```
x = 42 as float # 42.0
y = 3.14 as int  # 3 (ゼロ方向に切り捨て)
s = 42 as str    # "42"
b = true as int  # 1
```

よくあるミス: float から int へのキャストはゼロ方向に切り捨てるため、3.9 as int は4ではなく3になります。四捨五入が必要な場合は math モジュールの round() を使ってください。

再代入のルール

@const なしで宣言した変数は再代入できます。ただし、以下の制限があります:

```
x = 10
x = 20          # OK: 同じ型への再代入
# x = "text"   # エラー: 型を変更する再代入は禁止
```

@const は再代入できません。

```
@const
N = 5
# N = 6   # エラー: @const 変数への再代入は禁止
```

同名の変数を再宣言することもできません。

```
x = 1
# 同一スコープでの同名の再宣言は禁止
```

タプル分割代入

タプルを複数の変数に一度に展開できます。

```
@const
a, b = (10, 20)
print(a)    # 10
print(b)    # 20
```

ワイルドカード

_ を使って特定の位置の値を無視できます。

```
@const
x, _ = (1, 2)    # x のみが束縛される; 2 は破棄
print(x)        # 1
```

可変変数での分割代入

@const を省略すると可変変数として宣言できます。

```
a, b = (10, 20)
a = 99
print(a)    # 99
```

ルール

- 左辺の変数の数はタプルの要素数と一致させる必要があります。
 - 各変数は通常の @const/可変宣言と同じルールに従います。
 - ネストされたタプルの分割代入はサポートされていません。
-

<- 01 - はじめに / 次-> 03 - 演算子

[English](#) | [日本語](#) | [繁體中文](#)

03 - 演算子

<- 02 - 変数と型 / 次-> 04 - 制御構文

算術演算子

演算子	説明	例	結果
+	加算	3 + 2	5
-	減算	3 - 2	1
*	乗算 / 文字列繰り返し	3 * 2	6
/	除算 (常に float)	7 / 2	3.5
//	整数除算 (int 同士なら int、片方が float なら float)	7 // 2	3
%	剰余	7 % 3	1
**	累乗 (常に float)	2 ** 10	1024

```
a = 10
b = 3
```

```
print(a + b)    # 13
print(a - b)    # 7
print(a * b)    # 30
print(a / b)    # 3.3333... (float)
print(a // b)   # 3 (int)
print(a % b)    # 1
print(2 ** 8)   # 256 (float)
```

オーバーフロー安全性: int の算術演算 (+、-、*、単項-) は結果が 64 ビット符号付き範囲をオーバーフローするとランタイムエラーを発生させます。オーバーフローする定数式はコンパイル時に検出されます。低レベル型 (i32、u8 など) はサイレントにラップアラウンドします - 明示的なオーバーフロー制御には checked_add/saturating_add/wrapping_add を使ってください。

比較演算子

比較演算子はすべて bool 値を返します。

演算子	説明	例
==	等しい	a == b
!=	等しくない	a != b
<	より小さい	a < b
<=	以下	a <= b
>	より大きい	a > b
>=	以上	a >= b

```
x = 5
y = 10
```

```
print(x == y)    # false
print(x != y)    # true
```

```
print(x < y)      # true
print(x <= y)     # true
print(x > y)      # false
print(x >= y)     # false
```

文字列にも比較演算子が使えます（辞書順比較）。

```
print("abc" == "abc") # true
print("abc" < "abd")  # true
print("b" > "a")      # true
```

論理演算子

演算子	説明	例
and	論理 AND	a and b
or	論理 OR	a or b
not	論理 NOT	not a

```
t = true
f = false
```

```
print(t and f) # false
print(t or f)  # true
print(not t)   # false
print(not f)   # true
```

ビット演算子

ビット演算子は int 型にのみ使用できます。

演算子	説明	例
&	ビット AND	5 & 3 -> 1
\	ビット OR	5 \ 3 -> 7
^	ビット XOR	5 ^ 3 -> 6
~	ビット NOT (単項)	~5 -> -6
<<	左シフト	1 << 3 -> 8
>>	算術右シフト	8 >> 2 -> 2
>>>	論理右シフト	-1 >>> 1 -> 9223372036854775807

```
a = 0b1010 # 10
b = 0b1100 # 12
```

```
print(a & b) # 8 (0b1000)
print(a \| b) # 14 (0b1110)
print(a ^ b) # 6 (0b0110)
print(~a) # -11
print(1 << 4) # 16
print(32 >> 2) # 8
```

複合代入演算子

変数の値を更新する際に使える省略記法です。

演算子	説明	同等の式
<code>+=</code>	加算代入	<code>x = x + n</code>
<code>-=</code>	減算代入	<code>x = x - n</code>
<code>*=</code>	乗算代入	<code>x = x * n</code>
<code>/=</code>	除算代入	<code>x = x / n</code>
<code>%=</code>	剰余代入	<code>x = x % n</code>
<code>//=</code>	整数除算代入	<code>x = x // n</code>
<code>**=</code>	累乗代入	<code>x = x ** n</code>
<code>&=</code>	ビット AND 代入	<code>x = x & n</code>
<code> =</code>	ビット OR 代入	<code>x = x n</code>
<code>^=</code>	ビット XOR 代入	<code>x = x ^ n</code>
<code><<=</code>	左シフト代入	<code>x = x << n</code>
<code>>>=</code>	右シフト代入	<code>x = x >> n</code>

```
x = 10
x += 5    # x == 15
x -= 3    # x == 12
x *= 2    # x == 24
x /= 4    # x == 6 (float になる)
```

インクリメント・デクリメント演算子

変数を 1 増減させるための省略記法です。

演算子	説明	同等の式
<code>x++</code>	1 を加算	<code>x = x + 1</code>
<code>x--</code>	1 を減算	<code>x = x - 1</code>

```
count = 0
count++    # count == 1
count++    # count == 2
count--    # count == 1
```

注意: ステートメントとしてのみ使用可能で、式の中では使えません。

型昇格ルール

演算において `int` と `float` が混在する場合の挙動を以下に示します。

```
# + - * は片方が float なら結果は float
print(1 + 2)    # 3 (int)
print(1 + 2.0)  # 3 (float)
print(1.0 + 2)  # 3 (float)

# / は常に float
print(4 / 2)    # 2 (float)
```

```
# // は両方 int なら int、片方でも float なら float
print(7 // 2)      # 3 (int)
print(7.0 // 2)    # 3 (float)

# ** は常に float
print(2 ** 3)      # 8 (float)

# % は両辺 int なら int、片方 float なら float
print(7 % 3)       # 1 (int)
print(7.5 % 2)     # 1.5 (float)

# + は両辺 str なら文字列結合
print("foo" + "bar") # "foobar"

# * は片方が str、もう片方が int なら文字列繰り返し
print("ab" * 3)     # "ababab"
print(3 * "ab")     # "ababab"
```

所属演算子

演算子	説明	例
in	所属チェック	2 in {1, 2, 3} -> true
not in	否定の所属チェック	4 not in {1, 2, 3} -> true

```
s = {1, 2, 3}
print(2 in s)      # true
print(4 not in s)  # true
```

<- 02 - 変数と型 / 次-> 04 - 制御構文

[English](#) | [日本語](#) | [繁體中文](#)

制御構文

<- 前: 演算子 | 次: 関数 ->

if / else

条件に応じて処理を分岐させるには if を使います。

```
x = 10
```

```
if x > 0:
    print(x)
else:
    print(0)
```

- else は省略できます。
- 条件式には bool 以外も指定できます。int の場合、0 が偽、非 0 が真として扱われます。
- if はネストできます。

```

a = 5
b = 3

if a > 0:
    if b > 0:
        print(a + b)    # 8

```

case

`case:` は対象を持たない多分岐の条件分岐に、`case value:` は対象を持つパターンマッチングに使用します。いずれの形式も旧来の `when / match` キーワードを置き換えます。

条件分岐 `case:`

```

x = -2

case:
    x > 0:
        print("positive")
    x < 0:
        print("negative")
    -:
        print("zero")

```

複数の分岐をチェーンする場合に推奨される形式です。

パターン分岐 `case value:`

```

enum Color:
    Red
    Green
    Blue

c = Color::Green
case c:
    Color::Red:
        print("red")
    Color::Green:
        print("green")
    Color::Blue:
        print("blue")

```

`enum`、`Option`、`Result`、リテラルパターンを安全に分解するにはこの形式を使用します。

`case:` 式

`case:` は式としても使えます。 `_ =>` ワイルドカードアームが必須です。

```

label = case:
    score >= 90 => "A"
    score >= 80 => "B"
    _ => "C"

```

ネストされた三項式を置き換え、多分岐の値選択を読みやすく保ちます。

case value: 式

case value: も => を使って各アームから値を返す式として使えます。

```
res = case x:
    Some(v) => v
    None    => 0

label = case direction:
    Direction::North => "N"
    Direction::South => "S"
    Direction::East  => "E"
    Direction::West  => "W"
```

```
category = case score:
    n if n >= 90 => "A"
    n if n >= 80 => "B"
    -            => "F"
```

case 式は case 文と同じすべてのパターンをサポートします: リテラル、変数、enum、Option、Result、OR パターン (|)、ガード (if)、ワイルドカード (_)。case は網羅的でなければなりません。

if 式

値を生成するシンプルな 2 分岐の条件分岐には if 式を使います:

```
abs_val = if x >= 0 => x else -x
label = if score >= 90 => "A" else "B"
```

else ブランチは必須で、両方のブランチは同じ型を返す必要があります。多分岐の式には代わりに case: を使ってください。

while ループ

条件が真である間、ブロックを繰り返し実行します。

```
i = 3
while i > 0:
    print(i)
    i = i - 1
# 3
# 2
# 1
```

for ループと range

リストまたは range を使ってイテレーションできます。

```
for x in [1, 2, 3]:
    print(x)
# 1
# 2
# 3
```

range(n) は 0 から n - 1 までの整数を生成します。

```

for i in range(5):
    print(i)
# 0
# 1
# 2
# 3
# 4

```

range(start, end) は start から end - 1 までの整数を生成します。

```

for i in range(2, 5):
    print(i)
# 2
# 3
# 4

```

.. 範囲演算子は両端を含む範囲を生成します: 1 .. 3 は [1, 2, 3] を生成します。

```

for i in 1 .. 3:
    print(i)
# 1
# 2
# 3

```

for k, v in map でマップのキーと値のペアを走査できます:

```

m = {"x": 10, "y": 20}
for k, v in m:
    print(k)
    print(v)

```

break と continue

break はループを即座に抜けます。continue は現在の反復をスキップして次の反復へ進みます。

```

for i in range(10):
    if i == 5:
        break
    if i % 2 == 0:
        continue
    print(i)
# 1
# 3

```

while でも同様に使用できます。

```

n = 0
while true:
    n = n + 1
    if n % 2 == 0:
        continue
    if n > 7:
        break
    print(n)
# 1
# 3

```

```
# 5
# 7
```

注意: ネストしたループの中では、`break` / `continue` は最も内側のループにのみ作用します。ループ外で使用するとコンパイルエラーになります。

ネストの例

`for` と `while` はネストできます。

```
for i in range(1, 4):
    for j in range(1, 4):
        if j == 2:
            continue
        print(i * 10 + j)

# 11
# 13
# 21
# 23
# 31
# 33
```

スコープのルール

制御構文のブロックはスコープを持ちます。

ブロックスコープ

ブロック内で宣言した変数はブロック外から参照できません。

```
if true:
    inner = 42
# ここで inner を参照するとコンパイルエラー
```

外側の変数への参照・再代入

ブロック内から外側の変数を参照・再代入できます。

```
count = 0
for i in range(5):
    count = count + i
print(count)    # 10
```

内側スコープでの再代入

ブロック内で変数に代入すると、外側の変数が変更されます (Python スタイルのスコーピング)。シャドーイングは行われず、内側の代入は同じ変数を変更します。

```
x = 1
if true:
    x = 99
    print(x)    # 99
print(x)        # 99
```

パターンマッチング

case value: は enum、Option、Result、リテラルに対して安全に分岐します。

```
enum Color:
  Red
  Green
  Blue

c = Color::Green
case c:
  Color::Red:
    print("red")
  Color::Green:
    print("green")
  Color::Blue:
    print("blue")
# green
```

Option のマッチング

case value: を使って Some と None の両方のケースを安全に処理します。

```
x: Option<int> = Some(42)
case x:
  Some(v):
    print(v)
  None:
    print("nothing")
# 42
```

ワイルドカードとリテラル

_ は何にでもマッチするワイルドカードパターンです。リテラル値（数値・文字列・真偽値）でもマッチできます。

```
n = 5
case n:
  0:
    print("zero")
  1:
    print("one")
  _:
    print("other")
# other
```

guard 節

if でガード条件を追加できます。

```
case n:
  x if x > 0:
    print("positive")
  x if x < 0:
    print("negative")
  _:
    print("zero")
```

注意: `case value:` はすべてのパターンを網羅する必要があります。enum はすべてのバリエント、Option は `Some` と `None` の両方、リテラルは `_` ワイルドカードが必要です。

[<- 前: 演算子](#) | [次: 関数](#) ->

[English](#) | [日本語](#) | [繁體中文](#)

関数

[<- 前: 制御構文](#) | [次: Record と列挙型](#) ->

基本的な関数定義

関数は `function` キーワードで定義します。引数の型は `name: type` の形式で宣言します。型を省略した場合、デフォルトで `any` になります。戻り値の型は `->` の後に指定します。

```
function add(a: int, b: int) -> int:
  return a + b
```

- 引数の型宣言を推奨します。省略した場合、型は `any` にデフォルト設定されます。
 - 戻り値型は `->` の後に指定します。
 - `return` 文で値を返します。
-

関数の呼び出し

定義した関数は名前と引数を指定して呼び出します。

```
function multiply(x: int, y: int) -> int:
  return x * y
```

```
result = multiply(3, 4)
print(result)    # 12
```

再帰関数

関数は自分自身を呼び出せます（再帰）。

```
function factorial(n: int) -> int:
  if n <= 1:
    return 1
  return n * factorial(n - 1)
```

```
print(factorial(5))    # 120
print(factorial(0))    # 1
```

関数オーバーロード

引数の数や型が異なる同名の関数を複数定義できます。

```
function add(a: int, b: int) -> int:
    return a + b

function add(a: float, b: float) -> float:
    return a + b

print(add(1, 2))          # 3
print(add(1.5, 2.5))      # 4
```

呼び出し時の引数の型に応じて、適切な関数が自動的に選択されます。

注意: 引数の型が同一で戻り値の型だけが異なる関数を定義するとコンパイルエラーになります。

戻り値型の省略 (Unit 型)

戻り値が不要な関数は -> を省略できます。この場合、関数は Unit 型を返します。

```
function greet():
    print(42)

greet()    # 42
```

引数なし・戻り値なしの最もシンプルな関数形式です。

デフォルト引数

引数にデフォルト値を設定できます。呼び出し側がその引数を省略すると、デフォルト値が使われます。

```
function greet(name: str, greeting: str = "Hello") -> str:
    return f"{greeting}, {name}"

print(greet("Alice"))          # Hello, Alice
print(greet("Bob", "Good morning")) # Good morning, Bob
```

複数のデフォルトパラメータも使えます:

```
function connect(host: str, port: int = 8080, timeout: int = 30) -> str:
    return f"{host}:{port} (timeout={timeout})"

print(connect("localhost"))          # localhost:8080 (timeout=30)
print(connect("localhost", 3000))     # localhost:3000 (timeout=30)
print(connect("localhost", 3000, 10)) # localhost:3000 (timeout=10)
```

なぜデフォルト引数なのか? 一般的なケースではシンプルな呼び出しを保ちつつ、必要に応じてカスタマイズできます- 複数のオーバーロードが不要になります。注意: デフォルト値のある引数は、デフォルト値のない引数の後に配置する必要があります。

ラムダ関数

ラムダ関数は式として関数を記述できます。単一式のラムダは (parameters) => expression の形式を、ブロックラムダは (parameters): の後にインデントされたブロックを続けます。どちらの場合も戻り値の型は自動的に推論されます。

単一式ラムダ

```
double = (x: int) => x * 2
print(double(5)) # 10

add = (a: int, b: int) => a + b
print(add(3, 4)) # 7
```

引数なしラムダ

```
answer = () => 42
print(answer()) # 42
```

複数行ラムダ

: の後に改行とインデントを追加して複数の文を記述できます。

```
abs = (x: int):
    if x < 0:
        return -x
    return x

print(abs(-5)) # 5
print(abs(3)) # 3
```

なぜラムダなのか? 短い使い捨ての関数に最適です – 特に `filter` や `map` のような高階関数の引数として使います (後述)。

クロージャ

ラムダ関数は定義されたスコープの変数をキャプチャできます。関数とそのキャプチャされた環境の組み合わせをクロージャと呼びます。

```
offset = 10
add_offset = (x: int) => x + offset
print(add_offset(5)) # 15
```

クロージャは変数を**値によって**キャプチャします – クロージャ作成後に元の変数を変更しても、クロージャのコピーには影響しません。

```
base = 10
f = (x: int) => x + base
base = 999
print(f(1)) # 11 (キャプチャされた値 10 を使用)
```

これは双方向に作用します – クロージャ内の変更も外側の変数に影響しません:

```
counter = 0
items = [1, 2, 3]
items.map((x: int):
    counter += x # クロージャのローカルコピーのみ変更
    return x
)
print(counter) # 0 (外側の変数は変更されない)
```

なぜ値によるキャプチャなのか? 安全性と予測可能性を保証します – クロージャ内の変更を心配せずに、現在のスコープだけを見て変数の値を常に推論できます。**なぜクロージャなのか?** 関数をその場で特殊化できます。例えば、1つのテンプレートから加算関数のファミリーを作成できます。

高階関数

他の関数を引数として取る関数を定義できます。関数型は `function(parameter_types) -> return_type` と記述します。

```
function apply(f: function(int) -> int, x: int) -> int:
    return f(x)
```

```
double = (x: int) => x * 2
print(apply(double, 3))           # 6
print(apply((n: int) => n + 1, 10)) # 11
```

値としての関数

名前付き関数も変数に束縛したり引数として渡したりできます - ラムダと同じように動作します。

```
function square(x: int) -> int:
    return x * x
```

```
# 名前付き関数を引数として渡す
print(apply(square, 4)) # 16
```

```
# 変数に束縛
sq = square
print(sq(5)) # 25
```

なぜ高階関数なのか？ 何をするかとどのようにするかを分離できます。同じ `apply` 関数がどんな変換にも使え、コードの再利用性が高まります。コレクションの `filter`、`map`、`reduce` でこのパターンを既に見ています。

UFCS (統一関数呼び出し構文)

UFCS を使うと、`f(a, b)` を `a.f(b)` と書けます。最初の引数がドットの前に移動し、メソッドチェーンスタイルが可能になります。

```
function add(a: int, b: int) -> int:
    return a + b
```

```
x = 1
print(x.add(2)) # add(x, 2) -> 3
```

チェーン呼び出し

UFCS は複数の呼び出しをチェーンする際に真価を発揮します - 内側から外側へではなく、左から右へ読めるようになります:

```
function double(n: int) -> int:
    return n * 2
```

```
# チェーン (自然に読める: "x を取り、2 を足して、2倍する")
print(x.add(2).double()) # 6
```

```
# 同等のネストされた呼び出し (読みにくい)
print(double(add(x, 2))) # 6
```

なぜ UFCS なのか? 深くネストされた関数呼び出しを読みやすい左から右へのパイプラインに変えます。xs.iter().filter(...).map(...).to_list() のようなイテレータチェーンで既にこれを見えています。

演習

1. **デフォルト引数:** format_price(amount: int, currency: str = "USD", decimals: int = 2) -> str 関数を書いて、価格をフォーマットしてください。format_price(42) と format_price(42, "EUR") の両方が動作することを確認してください。
 2. **高階関数:** apply_twice(f: function(int) -> int, x: int) -> int 関数を書いて、f を x に 2 回適用してください (つまり f(f(x)))。 (x: int) => x + 1 でテストし、apply_twice((x: int) => x + 1, 5) が 7 を返すことを確認してください。
 3. **UFCS チェーン:** inc(n: int) -> int (1 を加算) と triple(n: int) -> int (3 倍にする) を定義してください。UFCS を使って 5.inc().triple() と書き、結果が 18 であることを確認してください。
-

[<- 前: 制御構文](#) | [次: Record と列挙型 ->](#)

[English](#) | [日本語](#) | [繁體中文](#)

Record と列挙型

[<- 前: 関数](#) | [次: コレクションとイテレータ ->](#)

Record の定義

record キーワードで record を定義します。各フィールドは name: type の形式で記述します。

record Point:

```
x: int
y: int
```

Record はスタック上の値型です。

インスタンスの生成

record 名を関数のように呼び出してインスタンスを生成します。引数はフィールドの定義順に指定します。

```
p = Point(10, 20)
```

フィールドアクセス (ドット記法)

フィールドにはドット記法でアクセスします。

```
p = Point(10, 20)
print(p.x)    # 10
print(p.y)    # 20
```

Record は直接 print できます:

```
p = Point(10, 20)
print(p)    # Point(x: 10, y: 20)
```

フィールドへの代入

@const なしで宣言した可変変数のフィールドは再代入できます。

```
p = Point(10, 20)
p.x = 100
print(p.x)    # 100
```

注意: @const で宣言した変数のフィールドへの代入はコンパイルエラーになります。

関数の引数としての Record

Record を関数の引数として渡せます。

```
record Point:
  x: int
  y: int

function distance_x(a: Point, b: Point) -> int:
  return a.x - b.x

p1 = Point(10, 3)
p2 = Point(4, 7)
print(distance_x(p1, p2))    # 6
```

ネストした Record

Record のフィールドに別の record を使えます。

```
record Point:
  x: int
  y: int

record Line:
  start: Point
  end: Point

line = Line(Point(0, 0), Point(10, 5))
print(line.start.x)    # 0
print(line.end.x)      # 10
```

ドット記法をチェーンすることでネストしたフィールドにアクセスできます。

列挙型 (enum)

enum キーワードで列挙型を定義できます。各バリエントは名前付きの定数として扱われます。

定義

```
enum Color:
    Red
    Green
    Blue
```

使い方

バリエントには `::` でアクセスします。

```
c = Color::Red
print(c)    # Red
```

比較

`==` や `!=` でバリエントを比較できます。

```
case:
    c == Color::Red:
        print("red!")
    c == Color::Green:
        print("green!")
    -:
        print("blue!")
```

関数引数

関数の引数型として enum 名を使えます。

```
function describe(c: Color) -> str:
    if c == Color::Red:
        return "warm"
    return "cool"
```

関連データ付き列挙型 (ADT)

enum のバリエントは関連する値を持つことができます。これにより、単一の enum でさまざまな形状のデータを表現できます – 代数的データ型 (ADT) として知られるパターンです。

```
enum Shape:
    Circle(radius: float)
    Rectangle(width: float, height: float)
    Point
```

名前付きフィールドはドキュメント用です – 定義を自己記述的にします。名前なし構文 (`Circle(float)`) も有効です。

ADT バリエントの構築

フィールドが名前付きかどうかに関わらず、構築は常に位置指定です。

```
c = Shape::Circle(3.14)
r = Shape::Rectangle(4.0, 5.0)
p = Shape::Point
```


ADT バリエントのマッチング

case を使って ADT バリエントから関連データを取り出します。バインディングにはフィールド名ではなく任意の変数名を使います。これは[制御構文](#)で学んだパターンマッチングと直接つながります。

```
function describe(s: Shape) -> str:
  case s:
    Shape::Circle(r):
      return f"circle with radius {r}"
    Shape::Rectangle(w, h):
      return f"rectangle {w}x{h}"
    Shape::Point:
      return "point"

print(describe(Shape::Circle(3.14)))      # circle with radius 3.14
print(describe(Shape::Rectangle(4.0, 5.0))) # rectangle 4.0x5.0
```

なぜ ADT なのか? 「複数の形状のうちの 1 つ」であるデータを型安全にモデル化できます。コンパイラがパターンマッチング時にすべてのバリエントを処理しているか保証し、漏れをコンパイル時に検出します。

ジェネリック列挙型

enum は型パラメータを取ることができ、異なるペイロード型にわたって再利用可能になります。

```
enum MyOption<T>:
  MySome(T)
  MyNone
```

使い方

```
a = MyOption<int>::MySome(42)
b: MyOption<int> = MyOption<int>::MyNone
```

```
case a:
  MyOption::MySome(v):
    print(v)      # 42
  MyOption::MyNone:
    print("none")
```

注意: Ry の組み込み Option<T> と Result<T, E> 型はこれとまったく同じように動作します。[エラーハンドリング](#)で学びます。

演算子オーバーロード

function operator 構文でカスタム型に演算子を定義できます。これにより record が +, == などの演算子で自然に動作するようになります。

二項演算子

2 つのパラメータを取ります。

```
record Vec2:
  x: int
  y: int
```

```

function operator+(a: Vec2, b: Vec2) -> Vec2:
    return Vec2(a.x + b.x, a.y + b.y)

function operator==(a: Vec2, b: Vec2) -> bool:
    return a.x == b.x and a.y == b.y

v1 = Vec2(1, 2)
v2 = Vec2(3, 4)
v3 = v1 + v2
print(v3.x)      # 4
print(v1 == v2)  # false

```

単項演算子

1つのパラメータを取ります。

```

function operator-(v: Vec2) -> Vec2:
    return Vec2(-v.x, -v.y)

```

サポートされている演算子

カテゴリ	演算子
算術	+, -, *, /, %, **, //
比較	==, !=, <, <=, >, >=
ビット	&, \ , ^, ~, <<, >>, >>>
論理	and, or, not
所属	in
インデックスアクセス	[]
インデックス代入	[] =
関数呼び出し	()
型キャスト	as
複合代入	+=, -=, *=, /=, %=, //=, **=, &=, \ =, ^=, <<=, >>=

なぜ演算子オーバーロードなのか？ ドメイン型に自然な構文を与えます。Vec2 + Vec2 は vec2_add(a, b) より読みやすく、== があれば case や比較でシームレスに動作します。

演習

1. **ADT**: Dog(name: str)、Cat(name: str, indoor: bool)、Fish のバリエーションを持つ Animal enum を定義してください。case を使って各バリエーションの説明を返す describe(a: Animal) -> str 関数を書いてください。
2. **演算子オーバーロード**: amount: int と currency: str を持つ Money record を定義してください。同じ通貨の2つの Money 値を足すと、合計金額の新しい Money を返すように + をオーバーロードしてください。

[<- 前: 関数](#) | [次: コレクションとイテレータ](#) ->

[English](#) | [日本語](#) | [繁體中文](#)

コレクションとイテレータ

[<- 前: Record と列挙型](#) | [次: エラーハンドリング ->](#)

Ry には 4 種類のコレクション型があります: タプル、リスト、マップ、セット。

タプル

タプルは複数の値を一つにまとめた不変のデータ構造です。異なる型の要素を保持できます。

生成

```
t = (1, 3.14)
```

型アノテーション

```
t: (int, float) = (1, 3.14)
```

要素アクセス

.0, .1, ... のようにインデックスでアクセスします。

```
t = (1, 3.14)
print(t.0)    # 1
print(t.1)    # 3.14
```

関数の戻り値

複数の値を返したいときにタプルが便利です。

```
function swap(a: int, b: int) -> (int, int):
    return (b, a)
```

```
result = swap(1, 2)
print(result.0) # 2
print(result.1) # 1
```

制限事項

- 範囲外のインデックス（例: 要素数 2 のタプルに .2 でアクセス）はコンパイルエラーになります。
 - print にタプルを直接渡すとエラーになります。各要素を個別に渡してください。
-

リスト

リストは同じ型の要素を並べた可変長のデータ構造です。

生成

```
xs = [1, 2, 3]
```

型アノテーション

```
xs: List<int> = [1, 2, 3]
```

インデックスアクセス

```
print(xs[0])    # 1

i = 1
print(xs[i])    # 2
```

インデックス代入

```
xs[0] = 99
```

length

```
print(length(xs))    # 3
```

print

```
print(xs)    # [1, 2, 3]
```

for 走査

```
for x in xs:
    print(x)
```

関数引数

```
function first(xs: List<int>) -> int:
    return xs[0]
```

filter, map, sort

リストは filter、map、sort 操作をサポートしています。これらは元のリストを変更せず、新しいリストを返します。

```
xs = [1, 2, 3, 4, 5]
```

```
# filter: 条件に一致する要素だけを残す
greater_than_three = filter(xs, (x: int) => x > 3)
print(greater_than_three)    # [4, 5]
```

```
# map: 各要素を変換する
doubled = map(xs, (x: int) => x * 2)
print(doubled)    # [2, 4, 6, 8, 10]
```

```
# sort: 昇順ソート (デフォルト)
sorted = sort([3, 1, 2])
print(sorted)    # [1, 2, 3]
```

```
# UFCS (統一関数呼び出し構文) によるチェーン
# x.f(args) は f(x, args) と等価
result = xs.filter((x: int) => x > 1).map((x: int) => x * 10).sort()
print(result)    # [20, 30, 40, 50]
```

reduce, fold

reduce はリストを最初の要素から 1 つの値に集約します。fold は明示的な初期値を指定できます。

```
xs = [1, 2, 3, 4, 5]

# reduce: 最初の要素から開始
total = reduce(xs, (a: int, b: int) => a + b)
print(total)    # 15

# fold: 明示的な初期値を指定
total2 = fold(xs, 0, (a: int, b: int) => a + b)
print(total2)   # 15
```

any, all

any は述語を満たす要素が 1 つ以上あれば true を返します。all はすべての要素が満たす場合に true を返します。

```
xs = [1, 2, 3, 4, 5]

print(any(xs, (x: int) => x > 4))    # true
print(any(xs, (x: int) => x > 9))    # false

print(all(xs, (x: int) => x > 0))    # true
print(all(xs, (x: int) => x > 3))    # false
```

sum, min, max

```
xs = [3, 1, 4, 1, 5]
print(sum(xs))    # 14
print(min(xs))    # 1
print(max(xs))    # 5
```

first, last, is_empty

```
xs = [10, 20, 30]
print(first(xs))    # Some(10)
print(last(xs))     # Some(30)
print(is_empty(xs)) # false
```

enumerate, zip

enumerate は各要素にインデックスを付けます。zip は 2 つのリストを要素ごとに結合します。

```
xs = [10, 20, 30]
indexed = enumerate(xs)
# [(0, 10), (1, 20), (2, 30)]
for p in indexed:
    print(p.0)
    print(p.1)

ys = ["a", "b", "c"]
zipped = zip(xs, ys)
# [(10, "a"), (20, "b"), (30, "c")]
```

制限事項

- 全要素が同じ型である必要があります。異なる型を混在させることはできません。
- 空リスト [] はエラーになります。
- 範囲外アクセスはランタイムエラー (exit(1)) になります。

- 要素の型として `int`, `float`, `bool`, `str` をサポートしています。
-

マップ

マップはキーと値のペアを管理する連想配列です。

生成

```
m = {"a": 1, "b": 2}
```

型アノテーション

```
m: Map<str, int> = {"a": 1, "b": 2}
```

キーアクセス

```
print(m["a"])    # 1
```

挿入 / 更新

新規キーへの代入で挿入、既存キーへの代入で更新します。

```
m["c"] = 3      # 新規追加
m["a"] = 99     # 更新
```

length

```
print(length(m))    # 3
```

print

```
print(m)    # {a: 99, b: 2, c: 3}
```

has_key

キーが存在するか確認します。

```
print(has_key(m, "a"))    # true
```

keys, values

`keys` は全キーのリストを返します。`values` は全値のリストを返します。

```
m = {"a": 1, "b": 2, "c": 3}
print(keys(m))    # ["a", "b", "c"]
print(values(m))  # [1, 2, 3]
```

関数引数

```
function get_val(m: Map<str, int>, k: str) -> int:
    return m[k]
```

制限事項

- 全キーが同じ型、全値が同じ型である必要があります。
 - 空マップは型注釈が必要です。
 - 存在しないキーへのアクセスはランタイムエラー（`exit(1)`）になります。
-

セット

セットは同じ型の要素を重複なしで保持するコレクションです。

生成

```
s = {1, 2, 3}
```

型アノテーション

```
s: Set<int> = {1, 2, 3}
```

in 演算子

要素がセットに含まれるかを `in` 演算子で確認できます。

```
print(2 in s)    # true
print(5 in s)    # false
```

add / remove

```
add(s, 4)        # 要素追加
remove(s, 1)      # 要素削除
add(s, 2)         # 既に存在するため無視
```

length / print

```
print(length(s)) # 3
print(s)          # {2, 3, 4}
```

for 走査

```
for x in s:
    print(x)
```

空セット

空セットは型注釈が必要です。

```
empty: Set<int> = {}
```

制限事項

- 全要素が同じ型である必要があります。
 - 要素の型として `int`, `float`, `bool`, `str` をサポートしています。
-

イテレータ

イテレータはコレクションを遅延的に処理する方法を提供します。各ステップで中間リストを作成する代わりに、パイプラインを通じて要素を1つずつ処理します。

なぜ遅延イテレータなのか？ リストに対して直接 `filter` と `map` をチェーンすると、各ステップで新しい中間リストが生成されます。イテレータを使うと、要素はパイプライン全体を1つずつ通過します-中間的なメモリ割り当てがありません。大きなコレクションを処理する場合や、最初の数件の結果だけが必要な場合 (`take` を使用) に重要です。

生成と消費

コレクションに対して `iter()` を呼んでイテレータを取得し、`to_list()` で結果をリストに実体化します:

```
xs = [1, 2, 3]
ys = to_list(iter(xs))    # [1, 2, 3]
```

操作のチェーン

`filter`、`map`、`take` をチェーンしてパイプラインを構築できます。これは関数で学んだ UFCS チェーンスタイルを使います:

```
result = to_list(take(map(filter(iter([1, 2, 3, 4, 5])), (x: int) => x > 2), (x: int) => x * 2), 2))
print(result)    # [6, 8]
```

UFCS チェーンスタイル (等価):

```
result = [1, 2, 3, 4, 5]
    .iter()
    .filter((x: int) => x > 2)
    .map((x: int) => x * 2)
    .take(2)
    .to_list()
print(result)    # [6, 8]
```

より実践的な例 - スコアのリストを処理:

```
scores = [85, 42, 93, 67, 78, 55, 91]
```

合格スコア (≥ 60) のうち上位3つを取得し、ボーナスとして2倍にする

```
top_bonus = to_list(take(map(filter(iter(scores), (s: int) => s >= 60), (s: int) => s * 2), 3))
print(top_bonus)    # [170, 186, 134]
```

`next()` による手動イテレーション

`next()` は `Option` を返します - 次の要素がある場合は `Some(value)`、イテレータが使い尽くされた場合は `None`。 `Option` についてはエラーハンドリングで詳しく学びます。

```
it = iter([10, 20])
print(next(it))    # Some(10)
print(next(it))    # Some(20)
print(next(it))    # None
```

for ループ

イテレータは `for` ループで直接使えます:

```
for x in filter(iter([1, 2, 3]), (x: int) => x > 1):
    print(x)    # 2, 3
```


マップとセットのイテレーション

マップはキーと値のタプルを生成します。セットは個々の要素を生成します:

```
for k, v in iter({"a": 1, "b": 2}):
    print(f"{k} = {v}")

for x in iter({10, 20, 30}):
    print(x)
```

よくあるミス

- `to_list()` を忘れる: イテレータパイプラインだけでは何も実行されません – 遅延的です。 `to_list()`、`for` ループ、または `next()` で消費する必要があります。
- `to_list()` を早すぎる位置で呼ぶ: `filter()` の前に `to_list()` を置くと、すべての要素を先に実体化してしまうため、遅延評価の目的が損なわれます。

演習

1. **イテレータパイプライン**: `xs = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]` が与えられたとき、イテレータパイプラインを使って偶数の合計を計算してください。(ヒント: `.filter()` の後に `.to_list()` と `sum()` を使います。)
2. **手動イテレーション**: `[100, 200, 300]` のイテレータを作成し、`case` 式で `next()` の `Some` と `None` を処理してください。

[← 前: Record と列挙型](#) | [次: エラーハンドリング](#) →

[English](#) | [日本語](#) | [☑ 体中文](#)

エラーハンドリング

[← 前: コレクションとイテレータ](#) | [次: パッケージ](#) →

Ry にはエラーや値の不在に対処するための 3 つの補完的な戦略があります: **Option** (値が存在しない可能性がある)、**Result** (操作が失敗する可能性がある)、**契約による設計** (境界で不正な状態を防止する)。このチュートリアルでは 3 つすべてと、それぞれの使い分けを説明します。

Option 型

`Option<T>` は値が存在するかどうかを表す型です。2 つのバリエーションがあります: `Some(value)` と `None`。

```
x: Option<int> = Some(42)
print(x)      # Some(42)

y: Option<int> = None
print(y)      # None
```

値の取り出し

`case` を使って内部の値を安全に取り出し、`None` の場合を処理します。これは[制御構文](#)で学んだパターンマッチングを使用します:

```

case x:
  Some(v):
    print(v)    # 42
  None:
    print("nothing")

```

なぜ Option なのか? 値の不在を型システムで明示的にします。-1 のような「マジックナンバー」を返したり null をチェックしたりする代わりに、呼び出し側が None のケースを処理しなければなりません – コンパイラがそれを保証します。

Option に出会う場面

すでに Option の動作を見ています: `iterator.next()` は `Option<T>` を返し、各要素に対して `Some(element)` を、イテレータが使い尽くされると `None` を返します。

Result 型

`Result<T, E>` は失敗する可能性のある操作に使用します。成功時は `Ok(value)`、失敗時は `Err(error)` を返します。

```

function divide(a: int, b: int) -> Result<int, Error>:
  if b == 0:
    return Err(Error("division by zero"))
  return Ok(a // b)

```

case による Result の処理

```

r = divide(10, 0)
case r:
  Ok(v):
    print(v)
  Err(e):
    print(e.message)    # division by zero

```

? 演算子 (エラー伝播)

`Result` を返す関数から、同じく `Result` を返す別の関数を呼び出す際に、`?` を使ってエラーを自動的に伝播できます。値が `Ok` であればアンラップされ、`Err` であればその関数はそのエラーを即座に返します。

```

function safe_divide(a: int, b: int) -> Result<int, Error>:
  if b == 0:
    return Err(Error("division by zero"))
  return Ok(a // b)

function divide_and_add(a: int, b: int) -> Result<int, Error>:
  v = safe_divide(a, b)?    # b == 0 なら Err を早期リターン
  return Ok(v + 1)

```

これは以下と等価です:

```

function divide_and_add(a: int, b: int) -> Result<int, Error>:
  case safe_divide(a, b):
    Ok(v):
      return Ok(v + 1)
    Err(e):
      return Err(e)

```

? 演算子はボイラープレートを除去し、複数の失敗しうる操作を簡潔にチェーンできます:

```
function compute(a: int, b: int, c: int) -> Result<int, Error>:
  x = safe_divide(a, b)?
  y = safe_divide(x, c)?
  return Ok(y + 1)
```

and_then と map によるメソッドチェーン

複数の Result を返す操作をチェーンしたいが? を使えない場合 (例: Result を返さない関数の中)、and_then と map を使ってネストの深い case 文を避けることができます。

and_then – それ自体が Result を返す操作をチェーンします:

```
# case を3段階にネストする代わりに:
result = safe_divide(100, 2)
  .and_then((v: int) => safe_divide(v, 5))
  .and_then((v: int) => safe_divide(v, 2))

case result:
  Ok(v): print(v)      # 5
  Err(e): print(e.message)
```

map – Result のラッパーを変えずに Ok の値を変換します:

```
result = safe_divide(10, 2)
  .map((v: int) => v * 10)

case result:
  Ok(v): print(v)      # 50
  Err(e): print(e.message)
```

どちらのメソッドも Err で短絡評価します – チェーンのいずれかのステップが失敗すると、残りのクロージャを実行せずにエラーが伝播します。

and_then と map を1つのチェーンで混在させることもできます:

```
result = safe_divide(100, 10)
  .and_then((v: int) => safe_divide(v, 2))
  .map((v: int) => v + 100)
# Ok(105)
```

なぜ Result なのか? 例外を使わずにエラーハンドリングを明示的にします。型シグネチャがどの関数が失敗しうるかを正確に示し、? 演算子がコードを簡潔に保ちます。

よくあるミス: Result を返さない関数で? を使うとコンパイルエラーになります。? 演算子は戻り値の型が Result の関数内でのみ使用できます。

契約による設計

Ry は Eiffel スタイルの契約による設計をサポートしており、事前条件 (require)、事後条件 (ensure)、record の不変条件 (invariant) があります。Option と Result が実行時にエラーを処理するのに対し、契約は不正な状態の発生を防止します。

完全な仕様については[契約による設計リファレンス](#)を参照してください。

事前条件 (require)

require を使って、関数が呼ばれる際に真でなければならない条件を指定します:

```
function deposit(amount: int, balance: int) -> int:
  require:
    amount > 0
    balance >= 0
  new_balance: int = balance + amount
  return new_balance
```

事前条件が満たされない場合、プログラムは以下のメッセージで終了します:

```
Contract violation: require failed in deposit()
```

事後条件 (ensure)

ensure を使って、関数が返る際に真でなければならない条件を指定します。戻り値はユーザーが選択した変数名に束縛されます:

```
function abs(x: int) -> int:
  ensure v:
    v >= 0
  if x < 0:
    return -x
  return x
```

Ry では関数の引数是不変なので、ensure ブロック内で直接参照できます:

```
function increment(x: int) -> int:
  ensure v:
    v == x + 1
  return x + 1
```

タプルの戻り値には複数の変数名を使います:

```
function divide(a: int, b: int) -> (int, int):
  ensure q, r:
    q >= 0
    r >= 0
  return (a // b, a % b)
```

require と ensure の組み合わせ

両方を同時に使えます。require は ensure より前に書く必要があります:

```
function deposit(amount: int, balance: int) -> int:
  require:
    amount > 0
    balance >= 0
  ensure v:
    v >= 0
    v == balance + amount
  new_balance: int = balance + amount
  return new_balance
```

Record の不変条件 (invariant)

invariant を使って、record に対して常に成り立つべき条件を指定します。不変条件はコンストラクション後とフィールド代入のたびにチェックされます:

```
record BankAccount:
  balance: int
  min_balance: int
```

```

invariant:
    balance >= min_balance

a = BankAccount(100, 0)    # OK: 100 >= 0
# a.balance = -1          # Contract violation: invariant failed

```

なぜ契約なのか？ 仮定をコード内に直接文書化し強制します。関数が `amount > 0` を要求する場合、契約が呼び出し側で即座に違反を検出します - 関数本体の奥深くで症状が現れるのではなく。

契約のルール

- `require` と `ensure` ブロックは任意で、関数本体の前に記述します。
- 両方使う場合、`require` は `ensure` より前に書く必要があります。
- `ensure` には変数束縛が必要です (例: `ensure v:`)。これにより戻り値に名前を付けます。
- `invariant` は `record` 定義の末尾、すべてのフィールド宣言の後に記述します。
- すべての契約違反は `exit(1)` で終了します。

使い分けの指針

戦略	使用場面	例
Option	値が正当に不在である場合	キーの検索、 <code>iterator.next()</code>
Result	意味のあるエラーで操作が失敗しうる場合	ファイル I/O、パース、 ネットワーク呼び出し
契約	不正な入力がそもそも発生してはいけない場合 (プログラマのミス)	負の入金額、範囲外イン デックス

経験則: - 外部要因 (ユーザー入力、ファイルシステム、ネットワーク) で失敗しうる操作には **Result** を使う。
- 「何もない」が正常で期待される結果の場合は **Option** を使う。- プログラムのミスを早期に検出するには契約を使う - これはアサーションであり、エラーハンドリングではない。

よくあるミス

1. **Result を無視する**: `Result` を返す関数を呼んで処理しないと、エラー情報が失われます。
2. **Result を返さない関数で ? を使う**: ? 演算子は囲む関数が `Result` を返す必要があります。
3. **Option と Result を混同する**: `Option` には `Some/None`、`Result` には `Ok/Err` があります。用途が異なります。

演習

1. **Result と ?**: ヘルパー関数を使って文字列を整数にパースし、2 倍にする関数 `parse_and_double(s: str) -> Result<int, Error>` を書いてください。エラー伝播には ? を使います。
2. **契約**: `amount > 0` かつ `amount <= balance` を `require` で、結果が非負であることを `ensure` で指定した関数 `withdraw(amount: int, balance: int) -> int` を書いてください。
3. **Option の処理**: `List<int>` を受け取り、最初の偶数を `Option<int>` として返す関数を書いてください。偶数が存在しない場合は `None` を返します。

[<- 前: コレクションとイテレータ](#) | [次: パッケージ ->](#)

[English](#) | [日本語](#) | [繁體中文](#)

パッケージ

[<- 前: エラーハンドリング](#) | [次: 並行処理 ->](#)

Ry はパッケージシステムを使って、ファイルやディレクトリにまたがるコードを管理します。詳細な仕様は [パッケージリファレンス](#) を参照してください。

from/import 構文

別ファイルの関数をインポートするには from 構文を使います。

```
from math import sqrt, PI    # 選択インポート
from math                    # 全インポート (すべての定義)
```

これで math.ry に定義された関数が使えるようになります。

サブディレクトリ (ドット区切り)

ドット区切りでサブディレクトリ内のパッケージを指定できます。

```
from utils.calc import add    # utils/calc.ry をインポート
```

ドット 1 つがディレクトリの区切りに対応します。

ディレクトリパッケージ

パッケージは単一の .ry ファイルでも、複数の .ry ファイルを含むディレクトリでも構いません。パッケージがディレクトリに解決される場合、その中のすべての .ry ファイルが自動的にロードされます。

```
mypackage/
  calc.ry      # function add(), function sub()
  string.ry    # function concat()

from mypackage          # add, sub, concat をインポート
from mypackage import add # add のみインポート
```

特別なエントリファイル (__init__.py のようなもの) は不要です。_ で始まるファイルは除外されます。

相対インポート

先頭の . を使って、現在のファイルのディレクトリからの相対パスでインポートします。テストファイルから兄弟モジュールをインポートする際に特に便利です。

```
from .helper import greet    # 同じディレクトリの helper.ry からインポート
from .utils import add       # utils/ サブディレクトリからインポート
from .utils.calc import mul   # utils/calc/ ネストされたサブディレクトリからインポート
from . import add, sub        # 現在のディレクトリパッケージからシンボルをインポート
```

相対インポートは現在のファイルのディレクトリに対してのみ解決されます。標準ライブラリやその他の検索パスは検索されません。プロジェクトに標準ライブラリパッケージと同名のモジュールがある場合の名前衝突を防ぎます。

```
# プロジェクトに src/math/stats.ry がある場合:
from .math import mean    # 常にローカルの math パッケージに解決される
from math import sqrt     # 標準ライブラリの math パッケージに解決される
```

注意: 親ディレクトリインポート (from ..) はサポートされていません。

標準ライブラリ (std)

std パッケージはすべてのプログラムに自動的にインポートされます。from std を記述する必要はありません。

```
# これらの関数はインポートなしで利用可能
print("hello")
n = length("world")
xs = range(5)
```

標準ライブラリのパッケージから特定の定義を明示的にインポートすることもできます:

```
from str import contains
```

RY_HOME

標準ライブラリは \$RY_HOME/share/std/ にインストールされます。RY_HOME のデフォルト値は ~/.ry です。

```
export RY_HOME="$HOME/.ry"    # デフォルト
```

検索パスの優先順位

パッケージファイルは以下の順序で検索されます:

1. インポート元ファイルのディレクトリ – インポートを記述したファイルと同じディレクトリを最初に探します。
 2. \$RY_HOME/share – 標準ライブラリの場所。旧来のインストールでは \$RY_HOME/lib にフォールバックします。
 3. 実行ファイル相対の share/ – ry 実行ファイルからの相対ディレクトリ。旧来のレイアウトでは実行ファイル相対の lib/ にフォールバックします。
 4. RY_PATH 環境変数 – 見つからない場合は RY_PATH に指定されたディレクトリを順番に検索します。
-

RY_PATH 環境変数

複数のディレクトリをコロン区切りで指定できます。

```
export RY_PATH=/home/user/ry-libs:/usr/local/ry-libs
```

設定後は、指定ディレクトリ内のパッケージをどこからでもインポートできます。

制限事項

- from 文はファイルのトップレベルにのみ記述できます。関数やブロックの内部には書けません。
- 同じパッケージを複数回インポートしても、自動的にスキップされます (二重インポートは発生しません)。

- 循環インポート（A が B をインポートし、B が A をインポートする）はエラーになります。

```
# エラー例: a.ry と b.ry が互いをインポートしている場合
# a.ry: from b import foo
# b.ry: from a import bar <- 循環インポートエラー
```

[<- 前: エラーハンドリング](#) | [次: 並行処理 ->](#)

[English](#) | [日本語](#) | [繁體中文](#)

並行処理

[<- 前: パッケージ](#) | [次: テスト ->](#)

Ry には並行・並列実行のための 3 つのモデルがあります: 軽量の I/O バウンドタスク向けの **async/await**、データ並列ループ向けの **@parallel**、きめ細かい制御が必要な CPU バウンドワークロード向けの **OS スレッド**。

async/await

`Task<T>` は並行処理のためのランタイムハンドルです。タスクを返す関数を宣言するには `async function` を使い、別の `async function` 内でタスクの完了を待つには `await` を、同期コンテキストからは `block_on(task)` を使います。

非同期関数の定義

```
async function add(a: int, b: int) -> int:
    return a + b
```

`async function` を呼ぶと即座に `Task<T>` が返されます - 処理はバックグラウンドで開始されます:

```
t: Task<int> = add(20, 22)
```

結果の待機

同期コードからは `block_on()` を使います:

```
print(block_on(t))           # 42
print(block_on(add(1, 2)))    # 3
```

非同期コードからは `await` を使います:

```
async function double_add(a: int, b: int) -> int:
    result = await add(a, b)
    return result * 2
```

```
print(block_on(double_add(3, 4))) # 14
```

非同期関数の合成

非同期操作を自然にチェーンできます:

```
async function fetch_score() -> int:
    return 42
```

```
async function process() -> str:
    score = await fetch_score()
    return f"Score: {score * 2}"
```



```
print(block_on(process())) # Score: 84
```

なぜ `async/await` なのか? 逐次的なコードのように読める並行コードを書けます。ランタイムがスレッドプール上で効率的にタスクをスケジューリングします- 待ち時間が大半を占める I/O バウンドな処理に最適です。

よくあるミス: 非 `async` 関数内で `await` を使うとコンパイルエラーになります。同期コンテキストからは代わりに `block_on()` を使ってください。

@parallel

`@parallel` ディレクティブはカウント付き `for` ループを複数の CPU コアにまたがって並列化します:

```
@parallel
for i in range(8):
    print(i)
```

制約事項

`@parallel for` は独立した反復のために設計されています。以下は拒否されます: - `break` と `continue` - 並列反復間には意味のある順序がありません。- **外側の可変変数への書き込み** - データ競合を引き起こします。

カウント付きループ (`range(...)` または整数の `..` レンジ) のみがサポートされています。

なぜ `@parallel` なのか? 独立した作業を並列化する最も簡単な方法です。ロックもアトミック操作も不要- ディレクティブを追加するだけで、ランタイムが分散を処理します。

OS スレッド

CPU バウンドなタスクや、きめ細かい同期が必要な場合は、`thread` モジュールを使って OS スレッドを作成します。

```
from thread import thread_spawn, thread_join, atomic_int_new, atomic_int_load, atomic_int_add
```

スレッドの生成

`thread_spawn` はクロージャを受け取り、新しい OS スレッドを開始します:

```
counter = atomic_int_new(0)

t = thread_spawn():
    atomic_int_add(counter, 1)
)

thread_join(t)
print(atomic_int_load(counter)) # 1
```

注意: キャプチャされた変数はスレッドにコピーされます。プリミティブ型 (`int`, `str` など) の場合、独立したコピーが生成されます。不透明ハンドル型 (`AtomicInt`, `Lock` など) の場合、基盤となるリソースが共有されます - これは同期のためにまさに望まれる動作です。

アトミック操作による共有状態

`AtomicInt` はロックなしでスレッドセーフな整数操作を提供します:

```

from thread import thread_spawn, thread_join, atomic_int_new, atomic_int_load, atomic_int_add

counter = atomic_int_new(0)

t1 = thread_spawn():
    atomic_int_add(counter, 1)
)
t2 = thread_spawn():
    atomic_int_add(counter, 1)
)

thread_join(t1)
thread_join(t2)
print(atomic_int_load(counter))    # 2

```

ロックによる相互排他

クリティカルセクション（単一のアトミック操作以上のもの）を保護する必要がある場合は、Lock を使います:

```

from thread import thread_spawn, thread_join, lock_new, lock_acquire, lock_release
from thread import atomic_int_new, atomic_int_load, atomic_int_add

lock = lock_new()
counter = atomic_int_new(0)

t = thread_spawn():
    lock_acquire(lock)
    # クリティカルセクション: 一度に 1 つのスレッドのみ
    atomic_int_add(counter, 1)
    lock_release(lock)
)

lock_acquire(lock)
atomic_int_add(counter, 1)
lock_release(lock)

thread_join(t)
print(atomic_int_load(counter))    # 2

```

その他の同期プリミティブ

thread モジュールは以下も提供しています:

プリミティブ	用途
RWLock	複数のリーダーまたは 1 つのライター（読み取りが多いワークロード向け）
Semaphore	リソースへの同時アクセスを制限
Barrier	N 個のスレッドが同じ地点に到達するまで待機
AtomicBool	スレッドセーフなブーリアンフラグ

完全な API については [Thread リファレンス](#) を参照してください。

ネットワーク (TCP ソケット)

Ry は `net` モジュールを通じて TCP ソケットをサポートしています。接続は失敗する可能性があるため、ネットワーク操作は `Result` 型 ([エラーハンドリング](#)参照) を返します。

`net` モジュールは接続管理用プリミティブを提供し、`send`、`receive`、`close` は言語ビルトイン (常にスコープ内 - `import` 不要) として任意のストリーム系ハンドルに対して動作します:

関数	提供元	説明
<code>bind(host, port)</code>	<code>net</code>	リスナーを確保します。 <code>Result<TcpListener, Error></code> を返します
<code>listen(listener, backlog)</code>	<code>net</code>	接続の受け入れを開始します。 <code>Result<Unit, Error></code> を返します
<code>accept(listener)</code>	<code>net</code>	次の接続を待機します。 <code>Result<TcpStream, Error></code> を返します
<code>connect(host, port)</code>	<code>net</code>	送信側ストリームを開きます。 <code>Result<TcpStream, Error></code> を返します
<code>send(stream, bytes)</code>	builtin	<code>List<u8></code> を送信します。 <code>Result<int, Error></code> を返します
<code>receive(stream, max)</code>	builtin	最大 <code>max</code> バイトを読み込みます。 <code>Result<List<u8>, Error></code> を返します
<code>close(handle)</code>	builtin	<code>TcpListener</code> 、 <code>TcpStream</code> 、または <code>TlsStream</code> を解放します

`import` が必要なのは `net` モジュールの関数だけです。`send` / `receive` / `close` はグローバルビルトインのため、`from net import send` を書くとコンパイルエラーになります。

非同期サーバーと同期クライアント間のエコー通信の例:

```
from net import bind, listen, accept, connect, listener_port
from io import to_bytes, bytes_to_str
```

```
async function echo_server(server: TcpListener) -> str:
    case accept(server):
        Ok(conn):
            case receive(conn, 4096):
                Ok(data):
                    case send(conn, data):
                        Ok(_):
                            ...
                        Err(e):
                            ...
                    Err(e):
                        ...
                close(conn)
            Err(e):
                ...
```

```

close(server)
return "done"

function run_client(port: int) -> str:
  case connect("127.0.0.1", port):
    Ok(conn):
      case send(conn, to_bytes("hello")):
        Ok(_):
          ...
        Err(e):
          ...
      case receive(conn, 4096):
        Ok(resp):
          case bytes_to_str(resp):
            Ok(msg):
              close(conn)
              return msg
            Err(e):
              ...
          Err(e):
            ...
        close(conn)
        return "fail"
      Err(e):
        return "fail"

  case bind("127.0.0.1", 0):
    Ok(server):
      case listen(server, 1):
        Ok(_):
          port = listener_port(server)
          t = echo_server(server)
          print(run_client(port))    # hello
          block_on(t)
        Err(e):
          print(e.message)
      Err(e):
        print(e.message)

```

TLS (`tls_connect`) やストリームごとのタイムアウトを含む完全な TCP API については[ネットワークリファレンス](#)を参照してください。

適切なモデルの選択

特徴	async/await	@parallel	thread モジュール
最適な用途	I/O バウンドタスク	データ並列ループ	CPU バウンド、きめ細かい制御
オーバーヘッド	低 (タスクスケジューリング)	低 (自動)	高 (OS スレッド生成)
同期複雑さ	await による自動同期 中	不要 (独立した反復) 低	手動 (Lock、Semaphore 等) 高

よくあるミス

1. `async function` の外で `await` を使う: 同期コンテキストからは代わりに `block_on()` を使ってください。
 2. `@parallel` 内で外側の変数に書き込む: データ競合を防ぐため、コンパイル時に拒否されます。
 3. `thread_join` を忘れる: メインプログラムがスレッド完了前に終了すると、スレッドの処理が失われる可能性があります。
 4. スレッド間でプリミティブ変数を共有する: プリミティブ型はクロージャにコピーされます。共有状態には `AtomicInt` や `Lock` を使ってください。
-

演習

1. `async/await`: 名前を返す `async function` と姓を返す `async function` の2つを書いてください。両方を `await` してフルネームを返す3つ目の `async function` を書き、`block_on()` で結果を表示してください。
 2. アトミック操作付きスレッド: 共有の `AtomicInt` にそれぞれ 10 を加算する5つのスレッドを生成してください。すべてのスレッドを `join` した後、カウンターが 50 であることを確認してください。
-

[← 前: パッケージ](#) | [次: テスト](#) →

[English](#) | [日本語](#) | [繁體中文](#)

テスト

[← 前: 並行処理](#) | [次: プロジェクトのビルド](#) →

Ry には `@describe`、`@it`、`expect` を使った RSpec スタイルの組み込みテスト構文があります。詳細な仕様は [テストリファレンス](#) を参照してください。

テストの実行

```
ry test                # *.test.ry ファイルを自動検出して実行
ry test tests/spec     # 指定ディレクトリ以下の *.test.ry を再帰的に実行
ry test tests/my_test.test.ry # 特定のテストファイルを実行
ry test -p             # 全テストを並列実行 (-p または --parallel)
```

すべてのテストが成功すると終了コード 0、1 つでも失敗すると 1 が返されます。

引数なしで実行すると、`ry test` は `package.toml` を探してプロジェクトルートを特定し、すべての `*.test.ry` ファイルを再帰的に検出します。

テストの書き方

名前付き関数に `@it` を付けることでテストケースを宣言し、関連するテスト群は `@describe` で注釈された関数でラップします。

```
@it("should add integers")
function test_add():
    expect(1 + 2).to_eq(3)

@describe("Calculator")
```

```
function calculator_tests():
  @it("should subtract integers")
  function test_sub():
    expect(5 - 3).to_eq(2)

  @it("should check booleans")
  function test_bool():
    expect(3 > 1).to_be_true()
```

- @it は説明文字列を取り、修飾された関数がテスト本体になります
- @describe はその本体内で定義された @it 関数をグループ化します。グループはネスト可能で、出力はネストの深さに比例してインデントされます
- @describe 関数の本体に直接宣言された変数は**共通セットアップ**として機能し、すべての内部の@it 関数から捕捉されます

```
@describe("shared setup")
function shared_setup_tests():
  base = 100
  offset = 5

  @it("should use base value")
  function test_base():
    expect(base).to_eq(100)

  @it("should use base and offset")
  function test_combined():
    expect(base + offset).to_eq(105)
```

- expect、mock、verify は ry test でのみ使用できます（通常の ry 実行ではコンパイルエラー）

旧来のラムダ形式: ラムダ引数を用いた describe("name", (): ...) と it("desc", (): ...) は引き続きパースされますが**非推奨**です。新しいコードでは名前付き関数に対するディレクティブ形式を優先してください。

マッチャー

マッチャー	説明	対応型
to_eq(expected)	等値比較	int, float, bool, str
to_not_eq(expected)	不等値アサーション	int, float, bool, str
to_be_true()	true アサーション	bool
to_be_false()	false アサーション	bool
to_be_none()	None アサーション	Option
to_be_some()	Option が Some であるアサーション	Option
to_be_ok()	Result が Ok であるアサーション	Result
to_be_err()	Result が Err であるアサーション	Result
to_contain(val)	コンテナに値が含まれるアサーション	List, Set, Map, str
to_not_contain(val)	コンテナに値が含まれないアサーション	List, Set, Map, str
to_be_greater_than(v)	actual > v アサーション	int, float

マッチャー	説明	対応型
<code>to_be_less_than(v)</code>	<code>actual < v</code> アサーション	int, float
<code>to_be_greater_than_or_eq(v)</code>	<code>actual >= v</code> アサーション	int, float
<code>to_be_less_than_or_eq(v)</code>	<code>actual <= v</code> アサーション	int, float
<code>to_have_length(n)</code>	長さが <code>n</code> に等しいアサーション	List, Set, Map, str
<code>to_be_empty()</code>	長さが 0 であるアサーション	List, Set, Map, str
<code>to_start_with(prefix)</code>	文字列がプレフィックスで始まるアサーション	str
<code>to_end_with(suffix)</code>	文字列がサフィックスで終わるアサーション	str

fail

`fail()` は現在のテストを即座に失敗としてマークします。

```
@it("should handle error")
function test_should_handle_error():
    result = find_user(-1)
    case result:
        Ok(v):
            fail("expected error")
        Err(e):
            expect(e.message).to_eq("not found")
```

- `fail()` - 汎用メッセージでテストを失敗にする
- `fail(message)` - カスタムメッセージでテストを失敗にする
- `ry test` モードでのみ使用可能

出力フォーマット

```
Calculator
+ should add integers
+ should subtract integers
- should check booleans
  line 10: expected true, got false
```

2 passed, 1 failed

+ は成功（緑）、- は失敗（赤）を示します。ネストした `@describe` グループは、ネストの深さに比例して内部テストをインデントします:

```
outer group
  inner group
    + should pass deeply nested test
```

モック

`mock(fn_name, replacement)`

現在の `@it` ブロック内で関数をモック実装に置き換えます。`@it` ブロック終了時に自動的に復元されます。元の関数の `require` と `ensure` の契約はモックされた呼び出しに対しても引き続き実行されます。

```
function fetch_data() -> str:
    return "real data"

@describe("mocking")
function mocking_tests():
    @it("should replace function")
    function test_replace():
        mock(fetch_data, () => "fake")
        expect(fetch_data()).to_eq("fake")

    @it("should auto-restore after it block")
    function test_restore():
        expect(fetch_data()).to_eq("real data")
```

`verify(fn_name)`

モックされた関数の呼び出し回数を返します。

```
@describe("verify")
function verify_tests():
    @it("should count calls")
    function test_count():
        mock(fetch_data, () => "fake")
        fetch_data()
        fetch_data()
        expect(verify(fetch_data)).to_eq(2)
```

パラメタライズドテスト

名前付き関数に `@each` と `@it` を組み合わせて、同じテストを複数の入力で実行できます:

```
@each([(1, 2, 3), (0, 0, 0), (-1, 1, 0)])
@it("should add {0} + {1} = {2}")
function test_add_each(a: int, b: int, expected: int):
    expect(a + b).to_eq(expected)
```

各タプルが個別のテストケースになります。説明文の `{0}`, `{1}` は実際の値で置換されます。関数のパラメータリストはタプルのアリティと一致している必要があります。

プロパティベーステスト

名前付き関数に `@property` と `@it` を組み合わせて、ランダム生成された入力でテストできます:

```
@property(count=100)
@it("should verify addition is commutative")
function test_add_commutative(a: int, b: int):
    expect(a + b).to_eq(b + a)
```


テストは `count` 回ランダム値で実行されます。Ry は各パラメータの型注釈からジェネレータを推論します。失敗時は反例が表示されます。

契約を使ったテスト

契約 ([エラーハンドリング](#)参照) はモックと連携します: 元の関数の `require` と `ensure` の契約はモックされた呼び出しに対しても引き続き実行されます。これにより、契約は暗黙的なテストアサーションとして機能します。

```
function deposit(amount: int, balance: int) -> int:
  require:
    amount > 0
  ensure v:
    v > balance
  return balance + amount

@describe("deposit")
function deposit_tests():
  @it("should enforce contracts even when mocked")
  function test_contract():
    mock(deposit, (amount: int, balance: int) => balance + amount)
    expect(deposit(10, 100)).to_eq(110)
    # deposit(-1, 100) は "require failed" で終了する
```

なぜこれが重要なのか: 実装の詳細をモックしつつ、契約のセーフティネットを保持できます。モックが事後条件に違反した場合、テストが即座にそれを検出します。

制限事項

- ネストは名前付き関数に対する `@describe` でのみサポートされています。旧来のラムダ形式 `describe("name", (): ...)` はネストできません
 - `before_each` / `after_each` はサポートされていません- 代わりに `@describe` 関数の本体に宣言する共通セットアップ変数を使ってください
 - オーバーロード関数および `@native` 関数はモックできません
-

演習

1. **基本的なテスト:** `max(a: int, b: int) -> int` 関数のテストとして、等しい値、正の数、負の数をカバーする `@describe` ブロックを書いてください。
 2. **モック:** 値を返す関数 `fetch_temperature() -> int` を書いてください。テスト内で固定値を返すようにモックし、`verify` で正確に 1 回呼ばれたことを確認してください。
 3. **パラメタライズドテスト:** `@each` を使って `is_even(n: int) -> bool` 関数を入力 [(2, true), (3, false), (0, true), (-4, true)] でテストしてください。
-

[<- 前: 並行処理](#) | [次: プロジェクトのビルド->](#)

[English](#) | [日本語](#) | [🇨🇳体中文](#)

プロジェクトを作る

← 前: テスト

このチュートリアルでは、**タスクトラッカー** — インメモリの To-Do リストを管理する小さなアプリケーションを構築します。このプロジェクトでは、これまで学んだ言語機能のほとんどを活用します:

- **Record** と **ADT enum** (データモデリング)
 - **コレクションとイテレータ** (タスクのフィルタリングと変換)
 - **エラーハンドリング** (Result, Option, 契約)
 - **F 文字列と UFCS** (読みやすい出力とメソッドチェーン)
 - **デフォルト引数付き関数** (柔軟な API)
 - **モジュール** (ファイル間のコード整理)
 - **テスト** (動作の検証)
-

プロジェクトのセットアップ

新しいプロジェクトを作成します:

```
ry new task-tracker
cd task-tracker
```

以下の構造が生成されます:

```
task-tracker/
  package.toml
  src/
    main.ry
```

ステップ 1: データモデルの定義

src/model.ry を作成し、タスクの Record とステータスの enum を定義します:

```
enum Status:
  Todo
  InProgress
  Done

record Task:
  id: int
  title: str
  status: Status
  invariant:
    length(title) > 0

function create_task(id: int, title: str, status: Status = Status::Todo) -> Task:
  require:
    length(title) > 0
  return Task(id, title, status)
```

ここでは複数の機能を同時に使っています: - **ADT enum** による Status ([Record](#) と [Enum](#) より) - **Record 不変条件** でタイトルが空でないことを保証 ([エラーハンドリング](#) より) - **デフォルト引数** で status のデフォルトを Todo に設定 ([関数](#) より) - **契約** (require) による構築時の安全チェック ([エラーハンドリング](#) より)

ステップ 2: タスク操作

タスクリストを操作する関数を `src/model.ry` に追加します:

```
function add_task(tasks: List<Task>, title: str) -> List<Task>:
  id = length(tasks) + 1
  task = create_task(id, title)
  append(tasks, task)
  return tasks

function find_task(tasks: List<Task>, id: int) -> Option<Task>:
  for t in tasks:
    if t.id == id:
      return Some(t)
  return None

function complete_task(tasks: List<Task>, id: int) -> Result<List<Task>, Error>:
  case find_task(tasks, id):
    Some(t):
      t.status = Status::Done
      return Ok(tasks)
    None:
      return Err(Error(f"task {id} not found"))

function pending_tasks(tasks: List<Task>) -> List<Task>:
  return tasks
    .iter()
    .filter((t: Task) => t.status == Status::Todo)
    .to_list()
```

使われているパターンに注目してください: - **Option** を `find_task` で使用 — タスクが存在しない可能性がある (**エラーハンドリング**より) - **Result** を `complete_task` で使用 — 存在しないタスクの完了はエラー - **イテレータパイプライン** と **UFCS** チェーンを `pending_tasks` で使用 (**コレクション** と **関数** より) - **F** 文字列をエラーメッセージで使用 (**変数と型**より)

ステップ 3: 表示

表示関数を `src/model.ry` に追加します:

```
function format_task(t: Task) -> str:
  marker = "[ ]"
  case t.status:
    Status::Todo:
      marker = "[ ]"
    Status::InProgress:
      marker = "[~]"
    Status::Done:
      marker = "[x]"
  return f"{marker} {t.id}. {t.title}"

function print_tasks(tasks: List<Task>):
  if is_empty(tasks):
    print("No tasks.")
  return
```

```
for t in tasks:
    print(format_task(t))
```

ステップ 4: メインプログラム

src/main.ry を編集します:

```
from model import create_task, add_task, complete_task, pending_tasks, print_tasks, Status, Task
```

```
tasks: List<Task> = []
```

```
# Add some tasks
```

```
tasks = add_task(tasks, "Buy groceries")
```

```
tasks = add_task(tasks, "Write documentation")
```

```
tasks = add_task(tasks, "Review pull request")
```

```
print("All tasks:")
```

```
print_tasks(tasks)
```

```
# Complete a task
```

```
case complete_task(tasks, 1):
```

```
    Ok(updated):
```

```
        tasks = updated
```

```
    Err(e):
```

```
        print(f"Error: {e}")
```

```
print("\nPending tasks:")
```

```
print_tasks(pending_tasks(tasks))
```

実行します:

```
ry src/main.ry
```

期待される出力:

```
All tasks:
```

```
[ ] 1. Buy groceries
```

```
[ ] 2. Write documentation
```

```
[ ] 3. Review pull request
```

```
Pending tasks:
```

```
[ ] 2. Write documentation
```

```
[ ] 3. Review pull request
```

ステップ 5: テストの作成

tests/model.test.ry を作成します:

```
from model import create_task, add_task, find_task, complete_task, pending_tasks, Status, Task
```

```
describe("Task model", ():
```

```
    it("creates a task with default status", ():
```

```
        t = create_task(1, "Test task")
```

```
        expect(t.status == Status::Todo).to_be_true()
```

```

    )

    it("adds tasks to a list", ():
        tasks: List<Task> = []
        tasks = add_task(tasks, "First")
        tasks = add_task(tasks, "Second")
        expect(length(tasks)).to_eq(2)
    )

    it("finds a task by id", ():
        tasks: List<Task> = []
        tasks = add_task(tasks, "Target")
        case find_task(tasks, 1):
            Some(t):
                expect(t.title).to_eq("Target")
            None:
                fail("expected to find task")
    )

    it("returns None for missing task", ():
        tasks: List<Task> = []
        expect(find_task(tasks, 99)).to_be_none()
    )

    it("completes a task", ():
        tasks: List<Task> = []
        tasks = add_task(tasks, "Do it")
        case complete_task(tasks, 1):
            Ok(updated):
                remaining = pending_tasks(updated)
                expect(is_empty(remaining)).to_be_true()
            Err(e):
                fail("unexpected error")
    )

    it("returns error for invalid id", ():
        tasks: List<Task> = []
        case complete_task(tasks, 999):
            Ok(_):
                fail("expected error")
            Err(e):
                expect(e.message != "").to_be_true()
    )
)

```

テストを実行します:

```
ry test
```

次のステップ

このプロジェクトを拡張する方法をいくつか紹介します:

- 期日の追加 — `Option<str>` を使ってオプションな期日フィールドを追加

- **JSON への永続化** — `json` モジュールを使ってタスクをファイルに保存・読み込み
- **優先度の追加** — `Status` を優先度レベルを持つ ADT に拡張
- **並行処理** — `@parallel` を使ってタスクのバッチを並列処理

これで Ry の基礎は固まりました。言語の完全な仕様については[リファレンスドキュメント](#)をご覧ください。

[← 前: テスト](#)